Chapter 6: Locking

xv6 RISC-V Study Notes

Based on Cox, Kaashoek & Morris — xv6: a simple, Unix-like teaching OS *(rev. 3, 2022)*

1 Big Picture: What Problem Does This Chapter Solve?

1.1 The Concurrency Problem

Modern kernels run on *multiple CPUs simultaneously*, all sharing the same physical memory. Three distinct sources introduce concurrency:

1. **Multiprocessor parallelism** — two CPUs execute kernel code at literally the same instant, both reading and writing the same data structures.
2. **Thread switching** — even on a uniprocessor, the scheduler can switch a thread mid-operation, and another thread may observe a half-updated structure.
3. **Interrupt handlers** — a device interrupt can fire at any instruction, running handler code that touches the same data as the interrupted thread.

Without a concurrency control mechanism, these scenarios produce **races** — bugs whose outcome depends on exact timing, making them nearly impossible to reproduce or debug reliably.

1.2 The Solution: Locks

A **lock** provides *mutual exclusion*: only one CPU may hold a given lock at a time. Any shared data item guarded by a lock can only be accessed while that lock is held, ensuring that concurrent modifications never overlap.

The price is **serialisation**: when two CPUs want the same lock, one must wait. This limits parallelism, so minimising unnecessary locking is a key performance concern.

Foundational Concept

What a lock really protects: A lock does not protect a data item directly — it protects the *invariants* of that data. Invariants are properties that must always be true when the data is not being modified (e.g., "every element's `next` field points to a valid list member"). A critical section may temporarily violate invariants during an update, but the lock guarantees no other CPU can observe that intermediate state.

Section Summary

Concurrency arises from multiprocessors, thread switching, and interrupts. It produces races — silent, timing-dependent bugs. Locks enforce mutual exclusion by serialising access to critical sections, ensuring that invariants are only ever observed in a valid state.

2 Races: What Goes Wrong Without Locks

2.1 The Classic Race: Concurrent List Push

Consider a singly-linked free list and a `push` operation that two CPUs call simultaneously:

```
void push(int data) {
    struct element *l = malloc(sizeof *l);
    l->data = data;
```

```
4     l->next = list;    // line 15: both CPUs read old list head
5     list = l;          // line 16: both CPUs write -- one is lost
6 }
```

1. CPU1 executes line 15: `l->next = list` (reads current head, say `A`).
2. CPU2 executes line 15: `l->next = list` (also reads `A` — same old head).
3. CPU1 executes line 16: `list = l1` (head now points to CPU1's new node).
4. CPU2 executes line 16: `list = l2` (overwrites CPU1's write — **l1 is lost from the list**).

Result: both new nodes point to `A`, but `l1` is unreachable. Memory leak and corrupted list.

2.2 Fixing It With a Lock

```
6  struct element *list = 0;
7  struct lock listlock;
8
9  void push(int data) {
10     struct element *l = malloc(sizeof *l);
11     l->data = data;
12     acquire(&listlock);    // enter critical section
13     l->next = list;
14     list = l;
15     release(&listlock);    // exit critical section
16 }
```

Lines 17–18 are now a *critical section*. Only one CPU can hold `listlock` at a time, so the two assignments are executed atomically with respect to each other. No CPU can observe the list with `list` still pointing to the old head while `l->next` already points there.

2.3 Key Terminology

Term	Definition
Race	A memory location is accessed concurrently, and at least one access is a write. Outcome depends on timing.
Critical section	The sequence of instructions between `acquire` and `release`.
Mutual exclusion	Property that only one CPU executes a critical section at a time.
Invariant	A property of a data structure that must hold when no operation is in progress.
Contention	Multiple CPUs competing for the same lock simultaneously.

Section Summary

A race occurs when two CPUs interleave accesses to shared data, producing a lost update or corrupted structure. Wrapping the vulnerable instructions in `acquire`/`release` turns

them into a critical section, ensuring atomicity with respect to other critical sections on the same lock.

3 Spinlock Implementation

3.1 Why a Simple Read-Test-Write Loop Fails

The naive implementation shown below is *broken* on a multiprocessor:

```
21 void acquire(struct spinlock *lk) {  // BROKEN
22     for(;;) {
23         if(lk->locked == 0) {    // line 25: both CPUs see 0
24             lk->locked = 1;      // line 26: both set 1 -- both think they hold the lock
25             break;
26         }
27     }
28 }
```

Two CPUs can simultaneously pass line 25 (both see `locked == 0`) and both set it to 1 on line 26, both believing they hold the lock. Mutual exclusion is violated.

3.2 The Fix: Atomic Read-Modify-Write

Lines 25 and 26 must be made a *single atomic step*. RISC-V provides the `amoswap r, a` instruction for exactly this:

Foundational Concept

`amoswap r, a` atomically swaps the value in register `r` with the value at memory address `a`. Special hardware ensures no other CPU can access address `a` between the read and the write. The old memory value is placed in `r`.
xv6's `acquire` uses the C library wrapper `__sync_lock_test_and_set`, which compiles to `amoswap`. It atomically writes 1 to `lk->locked` and returns the previous value. If the previous value was 0, we just acquired the lock. If it was 1, someone else holds it; we spin and retry.

3.3 acquire and release

Spinlock: acquire (kernel/spinlock.c:22)

1. Call `push_off()` — disable interrupts on this CPU (see §7).
2. Panic if the lock is already held by this CPU (detects re-entrant acquire).
3. Spin in a loop calling `__sync_lock_test_and_set(&lk->locked, 1)` until it returns 0, meaning we have swapped 0 out — we now hold the lock.
4. Issue a `__sync_synchronize()` memory barrier (see §8).
5. Record `lk->cpu = mycpu()` for debugging.

Spinlock: release (kernel/spinlock.c:47)

1. Panic if the lock is not held by this CPU.
2. Clear `lk->cpu = 0`.

3. Issue a `__sync_synchronize()` memory barrier.
4. Call `__sync_lock_release(&lk->locked)` — atomically writes 0, releasing the lock.
5. Call `pop_off()` — potentially re-enable interrupts on this CPU.

Section Summary

A naïve test-then-set acquire is broken on multiprocessors because the test and set are not atomic. RISC-V's `amoswap` combines them into one indivisible hardware operation. xv6 wraps this in `acquire`/`release`, adding interrupt disabling and memory barriers around the atomic swap.

4 Using Locks: Granularity and Guidelines

4.1 When to Lock

Two basic rules determine when a lock is needed:

1. If a variable can be written by one CPU while another CPU reads or writes it, use a lock.
2. If an invariant involves multiple memory locations, a *single* lock must protect all of them together so no CPU can observe a partial update.

4.2 Coarse-Grained vs. Fine-Grained Locking

	Coarse-grained	**Fine-grained**
Description	One lock for a large subsystem	Many locks, each covering a small resource
Parallelism	Low: only one CPU at a time	High: different CPUs work on different resources
Complexity	Low: easy to reason about	Higher: more locks, more risk of deadlock
xv6 example	`kmem.lock` (one lock for entire page allocator)	`ip->lock` (one lock per inode)
Extreme case	"Big kernel lock" — one lock for the whole kernel	Per-field locking

Lock placement matters too. Moving `acquire` earlier than necessary serialises more work (e.g., serialising `malloc` inside the push critical section is wasteful). Lock acquisition should be as close to the critical section as correctness permits.

4.3 All Locks in xv6 (Figure 6.3)

Lock	What It Protects
`bcache.lock`	Allocation of block buffer cache entries
`cons.lock`	Serialises console hardware access; prevents intermixed output
`ftable.lock`	Serialises allocation of `struct file` entries in the file table
`itable.lock`	Protects allocation of in-memory inode entries
`vdisk_lock`	Serialises disk hardware access and DMA descriptor queue
`kmem.lock`	Serialises kernel memory (page) allocation
`log.lock`	Serialises operations on the transaction log
`pipe pi->lock`	Serialises operations on each individual pipe
`pid_lock`	Serialises increments of `next_pid`
`proc p->lock`	Serialises changes to a process's state fields
`wait_lock`	Helps prevent lost wakeups in `wait`
`tickslock`	Serialises operations on the `ticks` counter
`inode ip->lock`	Serialises operations on each inode and its content
`buf b->lock`	Serialises operations on each individual block buffer

Section Summary

Every shared variable that can be concurrently written needs a lock. The granularity of locking is a trade-off: coarse locks are simple but limit parallelism; fine-grained locks allow more concurrency but raise the complexity and deadlock risk. xv6 uses a mix of both, with one notable example of each.

5 Deadlock and Lock Ordering

5.1 What Is a Deadlock?

Deadlock: The Classic Scenario

Two threads each need two locks, but acquire them in opposite orders:

	Thread T1	**Thread T2**
Step 1:	`acquire(A)`	`acquire(B)`
Step 2:	`acquire(B)` ← **waits**	`acquire(A)` ← **waits**

T1 holds A and waits for B. T2 holds B and waits for A. Neither can proceed. The system is stuck.

5.2 The Global Lock Ordering Rule

Foundational Concept

Global lock ordering: All code paths in the kernel that must hold multiple locks simultaneously must acquire them in the *same global order*. This is the only reliable way to prevent deadlock with spinlocks.
Lock ordering effectively makes locks part of each function's specification: callers must invoke functions in a way that respects the agreed-upon order.

5.3 Why Ordering Is Hard in Practice

Lock ordering becomes difficult when:

- The lock order conflicts with module boundaries (module M1 calls M2, but M2's lock must be acquired before M1's lock).
- Lock identities are not known in advance — e.g., the file system discovers locks as it traverses a path name, and the `wait`/`exit` code searches a process table.
- More fine-grained locking creates longer chains, increasing the chance of ordering conflicts.

5.4 xv6's Longest Lock Chains

The file system contains xv6's most complex lock ordering requirements. Creating a file requires simultaneously holding:

1. The directory lock
2. The new file's inode lock
3. A disk block buffer lock
4. `vdisk_lock` (disk driver)
5. The calling process's `p->lock`

These must always be acquired in exactly this order to prevent deadlock.

Section Summary

Deadlock occurs when threads wait for each other's locks in a cycle. The only solution with spinlocks is a global acquisition order respected by all code paths. This order is a design constraint that must be documented and enforced; it becomes increasingly difficult as locking schemes grow more fine-grained.

6 Re-entrant Locks

Re-entrant (recursive) locks allow a thread that already holds a lock to acquire it again without deadlocking. They seem like a convenience, but xv6 deliberately avoids them.

Consider two functions `f` and `g`, both protecting `data` with `lock`, and `call_once()` is expected to run exactly once:

```
f() { acquire(&lock); if(data==0){ call_once(); h(); data=1; } release(&lock); }
```

```
g() { acquire(&lock); if(data==0){ call_once(); data=1; } release(&lock); }
```

If `h()` internally calls `g()`:

- **Non-re-entrant (xv6):** `g()` panics or deadlocks — immediately visible bug.
- **Re-entrant:** `g()` re-enters successfully. `call_once()` is invoked *twice* — a silent correctness bug.

Re-entrant locks break the intuition that a lock makes a critical section atomic with respect to other critical sections on the same lock.

xv6's choice: Non-re-entrant spinlocks. A deadlock is a loud, observable failure that forces the programmer to fix the lock ordering. A silent double-call is far harder to discover.

Section Summary

Re-entrant locks appear to solve some deadlock problems but undermine the atomicity guarantee that makes locks useful. xv6 uses non-re-entrant locks: an erroneous re-acquire causes a visible panic rather than a silent invariant violation.

7 Locks and Interrupt Handlers

7.1 The Interrupt-Deadlock Hazard

1. Kernel thread `sys_sleep` acquires `tickslock`.
2. A timer interrupt fires on the *same CPU*.
3. The timer interrupt handler `clockintr` tries to acquire `tickslock`.
4. `clockintr` spins forever — `tickslock` is held by `sys_sleep`.
5. `sys_sleep` can never run again (its CPU is in the interrupt handler) — it can never release `tickslock`.
6. **Deadlock.** The CPU freezes.

7.2 xv6's Solution: Disable Interrupts on Acquire

Foundational Concept

Rule: A CPU must never hold a spinlock with interrupts enabled on *that same CPU*. xv6 enforces this by calling `push_off()` at the start of every `acquire`, which disables interrupts on the local CPU.

Note: interrupts on *other* CPUs are unaffected. An interrupt handler on another CPU can still spin-wait for a thread on a different CPU to release a lock — this is safe.

7.3 Nested Critical Sections: push_off / pop_off

Disabling interrupts must be *counted* to handle nesting correctly:

- `acquire` calls `push_off()` (`kernel/spinlock.c:89`), which saves the current interrupt-enable state and disables interrupts. A nesting counter is incremented.
- `release` calls `pop_off()` (`kernel/spinlock.c:100`), which decrements the counter. Only when the counter reaches zero are interrupts re-enabled (to the state saved at the outermost `push_off`).

Critical Ordering Constraint

`push_off()` must be called **before** `lk->locked` is set. If the order were reversed, there would be a brief window where the lock is held but interrupts are still enabled — a badly timed interrupt in that window would cause the deadlock described above.
Similarly, `pop_off()` must be called **after** `lk->locked` is cleared, never before.

Section Summary

Spinlocks and interrupt handlers on the same CPU can deadlock if the CPU holds a lock that the interrupt handler tries to acquire. xv6 prevents this by disabling interrupts on the current CPU for the entire duration any spinlock is held. Nested locks are handled with a push/pop counter that restores the original interrupt state at the outermost release.

8 Instruction and Memory Ordering

8.1 The Problem

It is natural to assume that instructions execute in the order written. This is *wrong* in multi-threaded code:

- **Compilers** reorder load/store instructions for efficiency (e.g., keeping values in registers, eliminating redundant stores).
- **CPUs** execute instructions out of order, speculatively, or in parallel when there are no data dependencies between them.

```
1 acquire(&listlock);
2 l->next = list;    // line 4: must happen before line 5
3 list = l;          // line 5
4 release(&listlock);
```

If the compiler or CPU moved line 4 to *after* `release` (line 6), another CPU could acquire the lock, see the updated `list`, and then find `l->next` still uninitialised. Catastrophic corruption.

8.2 Memory Barriers

Memory Barriers (`__sync_synchronize()`)

A **memory barrier** is an instruction that tells both the compiler and the CPU:

- Do not move any load or store instruction from one side of this barrier to the other.

xv6 inserts `__sync_synchronize()` in both `acquire` and `release`:

- The barrier in `acquire` ensures all loads/stores inside the critical section happen *after* the lock is acquired.
- The barrier in `release` ensures all loads/stores inside the critical section happen *before* the lock is released.

This prevents the lock-straddling reorderings that would violate mutual exclusion.

Section Summary

Compilers and CPUs both reorder memory operations for performance. Without barriers, instructions could move outside critical section boundaries, violating mutual exclusion even with a correct lock implementation. Memory barriers in `acquire` and `release` prevent all such reorderings.

9 Sleep Locks

9.1 The Spinlock Limitation

Spinlocks waste CPU time while spinning. This is acceptable for short critical sections (a few instructions), but becomes a serious problem when a lock must be held for a long time — for example, while waiting for a disk read that takes tens of milliseconds.

Two related problems make holding a spinlock for a long time illegal in xv6:

1. A process cannot `yield()` the CPU while holding a spinlock (which would allow scheduling to run — but the scheduler itself acquires locks, risking deadlock).
2. Interrupts must be disabled while holding a spinlock, so holding one for a long time prevents interrupt handling entirely.

9.2 Sleep Locks: The Solution

Sleep Lock Design (kernel/sleeplock.c:22)

A sleep lock has a `locked` field protected by an *internal spinlock*. The key difference:

- `acquiresleep(lk)`: if the lock is held, it calls `sleep()` — which atomically releases the spinlock and yields the CPU. The thread is blocked until the lock is free.
- `releasesleep(lk)`: sets `locked = 0` and calls `wakeup()` to wake any waiting thread.
- Interrupts remain **enabled** while holding a sleep lock.
- The CPU is free to **run other processes** while a thread waits for a sleep lock.

9.3 Spinlock vs. Sleep Lock: When to Use Which

Property	Spinlock	Sleep Lock
Waiting behaviour	Spins (wastes CPU)	Yields CPU (sleeps)
Interrupts while held	**Disabled**	Enabled
Can yield while held	**No**	Yes
Can use in interrupt handlers	Yes	**No**
Can use inside spinlock section	N/A	**No**
Best for	Short critical sections	Long I/O operations
xv6 example	`kmem.lock`, `cons.lock`	File system inode lock

Sleep Lock Restrictions

- Sleep locks **cannot** be used inside spinlock critical sections — `acquiresleep` may yield the CPU, violating the spinlock invariant.

- Sleep locks **cannot** be used in interrupt handlers — interrupt handlers cannot sleep.
- Spinlocks *can* be used inside sleep-lock critical sections.

Section Summary

Spinlocks are inappropriate when a lock must be held for a long time (e.g., disk I/O). Sleep locks solve this by yielding the CPU while waiting, keeping interrupts enabled and allowing other processes to run. The trade-off is that sleep locks cannot be used in interrupt handlers or inside spinlock critical sections.

10 Foundational vs. Implementation Details

10.1 Foundational Concepts (Must Understand)

Foundational Concepts

1. **Locks protect invariants, not data.** The lock ensures that no other CPU observes a data structure while its invariants are temporarily violated during an update.
2. **Atomic read-modify-write is the hardware foundation.** Without a single atomic instruction (like `amoswap`), no correct software lock is possible on a multiprocessor.
3. **Global lock ordering is the only deadlock-free strategy with spinlocks.** All code paths must acquire multiple locks in the same order. This constraint must be documented and enforced as part of each subsystem's interface.
4. **Spinlocks must disable interrupts on the holding CPU.** Otherwise, an interrupt handler that tries to acquire the same lock deadlocks the CPU.
5. **Memory barriers prevent compiler/CPU reordering across lock boundaries.** Without them, instructions can move outside critical sections, breaking mutual exclusion even with a correct lock.
6. **Two lock types serve two different use cases.** Spinlocks for short critical sections (safe in interrupt handlers); sleep locks for long-running operations (yield the CPU but cannot be used in interrupt context).

10.2 Implementation Details (Good to Know)

Implementation Detail

- `struct spinlock` (`kernel/spinlock.h:2`) has three fields: `locked` (0 or 1), `name` (debug string), `cpu` (which CPU holds it).
- `acquire` uses `__sync_lock_test_and_set`; `release` uses `__sync_lock_release` — both compile to `amoswap` on RISC-V.
- `push_off/pop_off` use a per-CPU nesting counter (`cpu->noff`) and a saved interrupt state (`cpu->intena`).
- `__sync_synchronize()` compiles to a RISC-V `fence` instruction, which enforces memory ordering at the hardware level.
- xv6's `kalloc.c` uses a single coarse-grained lock (`kmem.lock`); a per-CPU free list design with work-stealing would reduce contention.
- Re-entrant acquire is detected by checking if `lk->cpu == mycpu()` before spinning; xv6 panics in this case.

- `acquiresleep` (`kernel/sleeplock.c:22`) holds a spinlock internally while calling `sleep()`; `sleep` atomically releases the spinlock and puts the thread to sleep, avoiding a lost-wakeup window.

11 Real World Context

11.1 Lock Contention and Cache Costs

The performance cost of locks is not just spinning — it has a hardware dimension. When CPU A holds a lock, the lock variable is in A's cache. When CPU B tries to acquire it, the cache coherence protocol must transfer the cache line from A to B. This *cache line migration* can be orders of magnitude more expensive than a cache hit. High contention means frequent migration, which can be more costly than the actual work being serialised.

11.2 Alternatives to Locks

Technique	Description
Lock-free data structures	Use atomic instructions to build structures that require no locks (e.g., lock-free linked lists). Harder to reason about; require deep understanding of memory ordering.
Per-CPU data	Give each CPU its own copy of a data structure; eliminate sharing. xv6 could use per-CPU free lists to reduce `kmem` contention.
Pthreads (user space)	POSIX thread library provides user-level locks, barriers, and condition variables; requires OS support (sleeping blocked pthreads, TLB shootdown, etc.).

11.3 xv6 Limitations

- xv6 avoids lock-free programming for simplicity; locking is already complex enough.
- The coarse `kmem.lock` is a known bottleneck for allocation-heavy workloads.
- xv6 has no support for POSIX threads at the user level.

Chapter 6 — Complete Summary

Chapter 6 addresses the correctness challenges that arise when the kernel runs concurrently on multiple CPUs.

- **Races** occur when concurrent accesses to shared data overlap — they are silent, timing-dependent bugs.
- **Locks** prevent races by ensuring mutual exclusion. They protect the invariants of shared data structures.
- **Spinlocks** are built on `amoswap`, the RISC-V atomic swap instruction, wrapped with memory barriers (`__sync_synchronize`) and interrupt-disable bookkeeping (`push_off`/`pop_off`).
- **Deadlock** is prevented by a global lock acquisition order respected by all code paths.
- **Re-entrant locks** are avoided: a visible deadlock is safer than a silent double-entry bug.

- **Interrupts must be disabled** on a CPU while it holds any spinlock on that CPU, to prevent interrupt-handler deadlocks.
- **Memory barriers** in acquire/release prevent the compiler and CPU from reordering accesses outside critical section boundaries.
- **Sleep locks** complement spinlocks for long-held locks: they yield the CPU while waiting, keep interrupts enabled, but cannot be used in interrupt handlers or inside spinlock sections.